

Ministry of Higher Education and Scientific Research

École Nationale Polytechnique, Algiers



Department of Electrical Engineering

Power system analysis

Laboratory report

Development of a MATLAB toolbox:

**Y_{bus} and Z_{bus} matrices, load flow
and transient stability**

Submitted by:

LAGHA Khalil Ellah

Supervised by:

Prof. A. HELLAL

Academic year 2024–2025

Abstract

This report presents a MATLAB toolbox for power system analysis developed as part of the laboratory work of the course. It brings together the construction of the nodal admittance matrix $\mathbf{Y}_{\text{bus}}$ and the nodal impedance matrix $\mathbf{Z}_{\text{bus}}$, the solution of the load-flow problem by three numerical methods (Gauss–Seidel, Newton–Raphson and fast decoupled), and transient stability analysis using the classical model. The whole is driven by a graphical interface and backed by automated test scripts that reproduce every result presented here.

The methods are first derived, then evaluated on fourteen test systems ranging from 3 to 118 buses, including the IEEE 14, 30, 57 and 118 benchmark networks. Three main results emerge. First, the $\mathbf{Z}_{\text{bus}}$ built by successive branch addition agrees with $\mathbf{Y}_{\text{bus}}^{-1}$ to double-precision round-off (10^{-12} to 10^{-16}) on every network possessing a ground reference, including those with tap-changing transformers and parallel lines, which validates both construction paths against each other. Second, the three load-flow methods converge on all systems and agree to better than 2×10^{-4} p.u. in voltage magnitude, reproducing their respective theoretical behaviours quantitatively: quadratic convergence of Newton–Raphson in 3 to 5 iterations, cheap but more numerous iterations for the fast decoupled method, and linear convergence of Gauss–Seidel with a cost that grows with the size and the conditioning of the network. Third, the transient stability study of the 6-bus system places the critical clearing time between 0.45 and 0.50 s, consistent with the equal-area criterion.

Keywords: power systems, nodal admittance matrix, nodal impedance matrix, load flow, Gauss–Seidel, Newton–Raphson, fast decoupled load flow, transient stability, critical clearing time, MATLAB.

Contents

Nomenclature	iv
1 Introduction	1
1.1 Purpose of the work	1
1.2 Validation strategy	1
1.3 Outline	1
2 Program architecture and data model	1
2.1 General structure	1
2.2 Data model	2
3 Construction of the Y_{bus} and Z_{bus} matrices	3
3.1 Nodal admittance matrix	3
3.2 Nodal impedance matrix by the direct method	3
4 Load-flow methods	4
4.1 The problem to be solved	4
4.2 Gauss–Seidel method	4
4.3 Newton–Raphson method	5
4.4 Fast decoupled method	5
5 Transient stability analysis	5
5.1 Classical model	5
5.2 Fault sequence	6
5.3 Swing equation and integration	6
6 Results and discussion	6
6.1 Agreement of the direct Z_{bus} with Y_{bus}^{-1}	6
6.2 Load flow: convergence and cost	7
6.3 Detailed solution: IEEE 14-bus network	8
6.4 Transient stability and critical clearing time	9
7 Conclusion	11
A Built-in test systems	12
B File organisation	12

List of Figures

1	Graphical interface of the toolbox, developed in App Designer.	2
2	Top: iteration count of the three methods on the fourteen systems. Bottom: deviation $\max \mathbf{Z}_{\text{bus}} - \mathbf{Y}_{\text{bus}}^{-1} $ on a logarithmic scale for every network with a ground reference, all far below the numerical threshold of 10^{-10}	8
3	Transient stability of the 6-bus system: relative rotor angles $\delta_i - \delta_1$ after a fault at bus 6 cleared at $t_{cr} = 0.2$ s by opening line 4–6.	9
4	Relative angle $\delta_2 - \delta_1$ for different clearing times. The longer the fault persists, the larger the angular excursion of the first swing, up to the loss of synchronism between 0.45 and 0.50 s.	10

List of Tables

2	Deviation between the $\mathbf{Z}_{\text{bus}}$ built by branch addition and $\mathbf{Y}_{\text{bus}}^{-1}$	7
3	Convergence and cost of the three load-flow methods. The tolerance ε and the iteration limit are specific to each system; times are indicative, measured on a desktop machine.	7
4	IEEE 14-bus network: load-flow solution obtained by Newton–Raphson.	9
5	6-bus system, fault at bus 6: influence of the clearing time on the maximum angular excursion.	10
6	The fourteen test systems available in the toolbox.	12
7	Organisation of the MATLAB scripts.	12

Nomenclature

N	Number of buses	–
$\mathbf{Y}_{\text{bus}}$	Nodal admittance matrix	p.u.
$\mathbf{Z}_{\text{bus}}$	Nodal impedance matrix	p.u.
V_i, θ_i	Voltage magnitude and angle at bus i	p.u., rad
$S_i = P_i + jQ_i$	Complex power injected at bus i	p.u.
y	Series admittance of a branch	p.u.
B	Half line-charging susceptance	p.u.
a	Transformer tap ratio	–
$\mathbf{J}$	Jacobian matrix	–
$\mathbf{B}', \mathbf{B}''$	Constant matrices of the fast decoupled method	p.u.
E'	Transient internal EMF of a machine	p.u.
X'_d	Direct-axis transient reactance	p.u.
H	Inertia constant	s
δ	Rotor angle	rad or °
P_m, P_e	Mechanical and electrical power	p.u.
t_{cr}	Fault clearing time	s
ε	Convergence tolerance	–

Bus types. PQ (code 0): active and reactive power specified. PV (code 2): active power and voltage magnitude specified. Slack (code 1, also called the reference bus): voltage magnitude and angle specified.

1 Introduction

1.1 Purpose of the work

The analysis of a power system rests on a small number of computational tools that appear in every software package of the field: the description of the network by a nodal admittance matrix, its counterpart in the form of an impedance matrix, the solution of the steady state by a load-flow calculation, and the study of the dynamic behaviour following a fault. This laboratory work consists in implementing these tools, validating them, and comparing the numerical methods available to carry them out.

The program developed offers an interactive interface for computing the $\mathbf{Y}_{\text{bus}}$ and $\mathbf{Z}_{\text{bus}}$ matrices, solving the load flow by three numerical methods and running a transient stability analysis. Each function applies to one of the fourteen built-in test systems, from 3 to 118 buses, and returns its results as tables and plots.

1.2 Validation strategy

A power-system program produces numbers that are hard to verify by hand as soon as the network exceeds a few buses. The approach adopted here therefore relies on three independent levels of validation, and this is what justifies implementing several methods rather than one.

1. **Internal cross-validation.** The $\mathbf{Z}_{\text{bus}}$ is built by a direct method that never inverts $\mathbf{Y}_{\text{bus}}$. Comparing it with $\mathbf{Y}_{\text{bus}}^{-1}$ verifies both constructions simultaneously: they share no code and cannot plausibly be wrong in the same way.
2. **Cross-validation between methods.** The three load-flow methods solve the same set of equations by entirely different numerical routes. Their agreement on the bus voltages is strong evidence that the solution obtained really is the solution of the equations, independently of the algorithm used.
3. **External validation.** The IEEE 14, 30, 57 and 118 networks have published reference solutions. Comparing the losses and the voltage profile with those values checks the program as a whole, including the data entry.

1.3 Outline

Section 2 describes the program architecture and the data model. Section 3 deals with the construction of the $\mathbf{Y}_{\text{bus}}$ and $\mathbf{Z}_{\text{bus}}$ matrices. Section 4 presents the three load-flow methods and Section 5 the transient stability model. Section 6 collects and discusses all the results obtained on the fourteen test systems, and Section 7 concludes by returning to the limitations of the models adopted.

2 Program architecture and data model

2.1 General structure

The file `Main.m` is the entry point: it calls `programme.m`, which builds the App Designer interface shown in Figure 1. Three tools are offered: *Ybus & Zbus*, *Load flow* and *Transient stability*.

The interface drives the computation scripts in the base workspace through `evalin` and `assignin`: the data (`linedata`, `busdata`, `Ybus` and the results) are shared between scripts there. This choice has an important advantage for verification: the numerical core remains independent of the interface and can be run from the command line. The scripts `test_core.m` and `test_st.m`

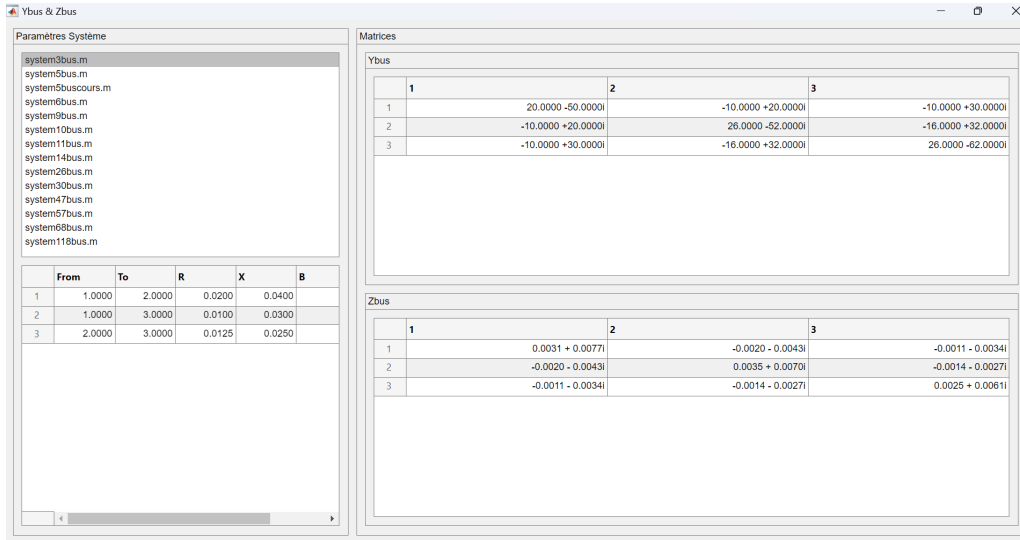


Figure 1. Graphical interface of the toolbox, developed in App Designer.

reproduce all the results of Section 6 without going through the interface, which makes the measurements reproducible and confirms that no result depends on a particular sequence of user actions.

Each tool has its own safeguards. The *Ybus & Zbus* tool displays the complex matrices in three formatted tabs ($\mathbf{Y}_{\text{bus}}$, direct $\mathbf{Z}_{\text{bus}}$ and $\mathbf{Y}_{\text{bus}}^{-1}$), with the indicator $\max |\mathbf{Z}_{\text{bus}} - \mathbf{Y}_{\text{bus}}^{-1}|$ permanently visible. The *Load flow* tool imposes a deterministic flat start, validates the inputs and displays an explicit convergence status rather than silently returning an unconverged result. The *Transient stability* tool integrates the fault parameters (T_{max} , t_{cr} , f , Δt) together with the choice of the faulted bus and of the line to be opened directly in its panel.

2.2 Data model

The network data follow the classical two-table format.

The `busdata` table has eleven columns:

$$[\text{Bus}, \text{Code}, |V|, \theta, P_L, Q_L, P_g, Q_g, Q_{\min}, Q_{\max}, Q_c]$$

where the **code** is 0 for a PQ bus, 2 for a PV bus and 1 for the slack bus.

The `linedata` table has six columns:

$$[n_l, n_r, R, X, B/2, \text{tap}]$$

Column 5 holds the *half* line-charging susceptance, and column 6 the transformer tap ratio, referred to the sending bus n_l .

One design point deserves mention: the slack bus is not always bus 1. It is bus 9 for the 47-bus system, bus 65 for the 68-bus system and bus 69 for the IEEE 118 network. The solvers therefore identify buses by their code and not by their position, which makes the implementation indifferent to that ordering. This precaution eliminates a whole class of silent errors in which a program works perfectly on small test networks and returns wrong answers on large ones.

3 Construction of the $\mathbf{Y}_{\text{bus}}$ and $\mathbf{Z}_{\text{bus}}$ matrices

3.1 Nodal admittance matrix

The nodal admittance matrix expresses Kirchhoff's laws in matrix form, $\mathbf{I} = \mathbf{Y}_{\text{bus}} \mathbf{V}$. Its construction is purely additive, each branch contributing to the corresponding diagonal and off-diagonal terms.

For a branch of series admittance $y = 1/(R + jX)$, with a charging shunt jB at each end and a tap a placed at the sending bus, the contribution is

$$Y_{ii} += \frac{y}{a^2} + jB, \quad Y_{jj} += y + jB, \quad (1)$$

$$Y_{ij} += -\frac{y}{a}, \quad Y_{ji} += -\frac{y}{a}. \quad (2)$$

The essential point is that contributions are **accumulated**, not assigned. This handles parallel lines correctly, as found in the IEEE 118 network, where two distinct branches connect the same pair of buses: an implementation using assignment would overwrite the first contribution with the second and underestimate the admittance of the connection.

```

1  for k = 1:nbr
2      i = linedata(k,1); j = linedata(k,2);
3      a = linedata(k,6); if a == 0, a = 1; end % no tap -> a = 1
4      y = 1/(linedata(k,3) + 1j*linedata(k,4)); % series admittance
5      ysh = 1j*linedata(k,5); % shunt = j*B (col. 5 = B/2)
6
7      Ybus(i,i) = Ybus(i,i) + y/(a*a) + ysh;
8      Ybus(j,j) = Ybus(j,j) + y + ysh;
9      Ybus(i,j) = Ybus(i,j) - y/a;
10     Ybus(j,i) = Ybus(j,i) - y/a;
11 end

```

Listing 1. $\mathbf{Y}_{\text{bus}}$ constructor, extracted from `Y_bus.m`. Contributions are accumulated, which handles parallel lines and transformer taps.

3.2 Nodal impedance matrix by the direct method

The script `Z_bus.m` builds $\mathbf{Z}_{\text{bus}}$ by **successive branch addition**, without ever inverting $\mathbf{Y}_{\text{bus}}$. One starts from a network reduced to the reference alone and adds branches one at a time, updating the matrix. Three elementary operations occur, depending on the nature of the branch being added.

Type 1 — new bus connected to the reference. The branch connects a new bus to ground through a shunt. The matrix is bordered with a zero row and column, with the branch impedance in the diagonal position.

Type 2 — new bus connected to an existing bus. The branch connects a new bus p to an existing bus r . The new row and column copy those of r , and the diagonal term is $Z_{rr} + z_{pr}$.

Type 3 — loop closure between two existing buses. The branch connects two existing buses a and b . The matrix is first bordered as in type 2, then the fictitious bus that was added is eliminated by Kron reduction:

$$\mathbf{Z}_{\text{bus}} \leftarrow \mathbf{Z}_{\text{bus}} - \frac{(\mathbf{z}_a - \mathbf{z}_b)(\mathbf{z}_a - \mathbf{z}_b)^T}{Z_{aa} + Z_{bb} - 2Z_{ab} + z_b}, \quad (3)$$

where $\mathbf{z}_a$ denotes column a of $\mathbf{Z}_{\text{bus}}$.

The line-charging shunts provide the ground reference required by type 1. **Tap-changing transformers are first folded into their equivalent π model**: a series impedance $z \cdot a$, that is, an admittance y/a , with a shunt leg $y(1-a)/a^2$ on the tap side and $y(a-1)/a$ on the opposite side, both legs being aggregated with the line shunts of each bus. The direct construction then applies to *any* network possessing at least one shunt element, including those with transformers and parallel lines.

There is, however, a physical rather than numerical limit to this construction. In the complete absence of shunt elements, as in the 3-bus system of the collection, the $\mathbf{Y}_{\text{bus}}$ matrix is singular: every row sums to zero, since no current can return through the ground. A ground-referred $\mathbf{Z}_{\text{bus}}$ then does not exist, and the tool falls back cleanly on the pseudo-inverse `pinv` while reporting the situation to the user, rather than returning a meaningless matrix.

4 Load-flow methods

4.1 The problem to be solved

The three methods solve the same set of power-balance equations, which at each bus read

$$S_i = P_i + jQ_i = V_i \overline{\left(\sum_{j=1}^N Y_{ij} V_j \right)}. \quad (4)$$

This system is nonlinear: the unknown complex voltages appear multiplied by their own conjugates. The effective unknowns are the angles θ_i of every bus except the slack, and the magnitudes $|V_i|$ of the PQ buses only.

The three methods differ in how they linearise or circumvent this nonlinearity, and it is that difference which explains their respective convergence behaviour.

4.2 Gauss–Seidel method

The Gauss–Seidel method rewrites (4) as a fixed-point iteration and updates the voltages successively, using the already updated values immediately:

$$V_i^{(k+1)} = \frac{1}{Y_{ii}} \left(\frac{\overline{S_i}}{\overline{V_i}} - \sum_{j \neq i} Y_{ij} V_j \right), \quad V_i \leftarrow V_i + \alpha (V_i^{(k+1)} - V_i), \quad (5)$$

where α is an acceleration factor.

The method is simple to program and its convergence is linear: the error is reduced by a roughly constant factor at each iteration. The number of iterations therefore grows with the size and the conditioning of the network, while the cost of one iteration remains very small.

The treatment of PV buses requires care. The reactive power needed to maintain $|V_i| = V_i^{\text{spec}}$ is evaluated at the set-point, following Stagg’s scheme, and then bounded by $[Q_{\min}, Q_{\max}]$. When the limit is reached, the bus is reclassified as PQ at the limiting value and acceleration is disabled for that bus; without this, repeated crossing of the limit produces a sustained oscillation. The convergence test is applied to the full phasor, $\max_i |V_i^{(k+1)} - V_i^{(k)}| < \epsilon$, not to the magnitude alone.

4.3 Newton–Raphson method

The Newton–Raphson method linearises (4) about the current point and solves at each iteration

$$\mathbf{J} \Delta \mathbf{x} = \Delta \mathbf{F}, \quad (6)$$

where $\Delta \mathbf{F} = [\Delta P; \Delta Q; \Delta |V|^2]$ collects the residuals.

The implementation adopted uses **rectangular coordinates** $V_i = e_i + jf_i$, so that the full Jacobian has six blocks: $\partial P/\partial e$, $\partial P/\partial f$, $\partial Q/\partial e$, $\partial Q/\partial f$, $\partial |V|^2/\partial e$ and $\partial |V|^2/\partial f$. This choice avoids trigonometric functions and makes the derivatives polynomial, hence exact and fast to evaluate, at the cost of one extra equation per PV bus to enforce the voltage magnitude.

Convergence is quadratic: the number of exact digits doubles at each iteration, which explains the 3 to 5 iterations observed regardless of network size. Each iteration is expensive, however, since assembling and solving the Jacobian dominates the computation time.

For the partitioning of the Jacobian, buses are reordered in the sequence [slack; PV; PQ], and the results are written back in the original order before display. The reactive limits of PV buses are applied by an outer loop: a saturated bus is switched to PQ at its limit and the load flow is re-run.

4.4 Fast decoupled method

The fast decoupled load flow exploits two properties of transmission networks: the strong predominance of reactance over resistance, and the small angular differences between neighbouring buses. Together they decouple the P – θ and Q – V subproblems:

$$\Delta \theta = [\mathbf{B}']^{-1} (\Delta P/V), \quad \Delta V = [\mathbf{B}'']^{-1} (\Delta Q/V). \quad (7)$$

The matrix $\mathbf{B}'$ is built from the line *reactances only*, following the BX scheme, and $\mathbf{B}'' = -\text{Im}(\mathbf{Y}_{\text{bus}})$ restricted to the PQ buses. Both matrices are **constant** and are factorised once before the iterative loop. The cost per iteration therefore reduces to a triangular solve, far below that of a Newton–Raphson iteration, at the price of more iterations since convergence is no longer quadratic.

The method has a limitation worth knowing: the decoupling assumptions degrade as the R/X ratio increases, which is the case in distribution networks. In the present implementation the reactive limits of PV buses are not enforced by the fast decoupled solver, which accounts for small discrepancies at saturated buses relative to the other two methods.

5 Transient stability analysis

5.1 Classical model

The script `st.m` implements the *classical* transient stability model. Each machine is represented by an internal EMF E' of constant magnitude behind its transient impedance $R_a + jX'_d$, and the loads are converted into constant admittances and merged into the $\mathbf{Y}_{\text{bus}}$ matrix.

The pre-fault state is obtained from a converged load flow, whose complex voltages initialise the internal EMFs and the rotor angles. This initialisation is essential: incorrect initial angles produce a spurious oscillation from the very first time step, independent of the fault being studied, which makes the result uninterpretable.

5.2 Fault sequence

The simulated sequence involves three successive network configurations.

1. **Pre-fault.** The nominal network, whose load flow provides the initial operating point.
2. **During the fault.** A solid short circuit at the chosen bus is modelled by forcing its voltage to zero, which defines the network $\mathbf{Y}_1$. The transmissible electrical power collapses while the mechanical power remains constant, so the machines accelerate.
3. **Post-clearing.** After the clearing time t_{cr} the faulted line is opened, which defines the network $\mathbf{Y}_2$. The modified topology transmits less power than the nominal network, hence a new equilibrium point.

5.3 Swing equation and integration

The equation of motion of each machine is

$$\frac{d\omega}{dt} = \frac{\pi f}{H} (P_m - P_e), \quad \frac{d\delta}{dt} = \omega - \omega_n. \quad (8)$$

It is integrated by the explicit Euler method. At each step the network voltages are solved by a Gauss sweep with the internal EMFs held fixed, which yields the electrical power P_e injected by each machine.

The choice of Euler's method calls for a comment. It has low accuracy and introduces a numerical damping that depends on the step size Δt , which makes it unsuitable for studying oscillations over several seconds. It remains acceptable for the purpose here, which is to determine whether the system survives its *first* swing, an event that takes place over a few hundred milliseconds. This is the same restriction as that of the equal-area criterion, against which the results of Section 6.4 are indeed compared.

6 Results and discussion

All results in this section were obtained under MATLAB R2024b in automated runs, using the scripts `test_core.m`, `test_st.m` and `report/make_results.m`, on the fourteen test systems.

6.1 Agreement of the direct $\mathbf{Z}_{bus}$ with $\mathbf{Y}_{bus}^{-1}$

Table 2 gives the deviation $\max |\mathbf{Z}_{bus}^{direct} - \mathbf{Y}_{bus}^{-1}|$ for each system, and the lower part of Figure 2 shows it on a logarithmic scale.

Discussion. The deviation lies everywhere between 10^{-12} and 10^{-16} , that is, at the level of double-precision round-off. The incremental construction, applied to fourteen systems with up to 186 branches, and the direct inversion therefore produce *the same matrix*. This result validates both computation chains at once: the $\mathbf{Y}_{bus}$ constructor of Listing 1, with its accumulation, its taps and its parallel lines, and the branch-addition algorithm of Section 3.2 with the folding of transformers into their π model. The value of this test lies in the fact that the two paths share no line of code, so an error common to both is very unlikely.

The 3-bus case illustrates the physical limit already announced. Without any shunt element, every row of $\mathbf{Y}_{bus}$ sums to zero, the matrix is singular, and the impedance “seen from ground” is simply not defined. That the tool detects and reports this, rather than returning the result of a numerically unstable inversion, is part of the result.

Table 2. Deviation between the $\mathbf{Z}_{\text{bus}}$ built by branch addition and $\mathbf{Y}_{\text{bus}}^{-1}$.

System	N	Transf.	$\mathbf{Z}_{\text{bus}}$ error	System	N	Transf.	$\mathbf{Z}_{\text{bus}}$ error
3	3	no	singular ¹	14	14	yes	1.1×10^{-13}
5	5	no	8.8×10^{-14}	26	26	yes	4.4×10^{-14}
5cours	5	no	8.8×10^{-14}	30	30	yes	1.7×10^{-13}
6	6	no	1.0×10^{-12}	47	47	no	2.4×10^{-14}
9	9	yes	2.5×10^{-15}	57	57	yes	7.9×10^{-14}
10	10	yes	4.6×10^{-14}	68	68	yes	7.4×10^{-16}
11	11	no	3.8×10^{-12}	118	118	no ²	7.4×10^{-15}

¹ no shunt element: $\mathbf{Y}_{\text{bus}}$ is singular and a ground-referred $\mathbf{Z}_{\text{bus}}$ does not exist; the fallback on pinv is reported.

² network containing parallel lines.

6.2 Load flow: convergence and cost

Table 3 compares the three methods in terms of iteration count, computation time and active losses.

Table 3. Convergence and cost of the three load-flow methods. The tolerance ε and the iteration limit are specific to each system; times are indicative, measured on a desktop machine.

System	Gauss–Seidel		Newton–Raphson		Fast decoupled		P_{loss} (MW)	losses (%)
	it.	t (ms)	it.	t (ms)	it.	t (ms)		
3	19	40	4	53	14	22	18.42	4.40
5	22	20	4	37	9	36	3.05	1.99
5cours	14	19	3	54	8	34	4.57	2.69
6	20	12	4	31	6	19	5.29	1.49
9	36	26	4	24	6	23	3.94	1.24
10	19	22	3	31	9	21	3.62	3.79
11	17	16	3	11	7	14	6.65	1.10
14	24	13	4	12	10	36	13.39	4.92
26	22	13	3	30	6	20	15.54	1.22
30	26	13	4	12	8	10	17.60	5.85
47	7	18	3	11	5	10	12.86	1.42
57	48	13	4	13	13	15	28.54	2.23
68	51	11	5	22	16	15	68.22	27.23 ¹
118	22	8	4	20	6	12	131.96	3.02

¹ The 68-bus system (NETS–NYPS aggregate) is heavily loaded; the percentage is referred to total generation.

Discussion. The theoretical behaviour of the three methods is recovered quantitatively.

- *Newton–Raphson* converges in 3 to 5 iterations regardless of network size, the signature of quadratic convergence. The iteration count is remarkably insensitive to size: 4 iterations for the 3-bus network as well as for the 118-bus one. Each iteration is expensive, however, because of the assembly of the full Jacobian.
- *The fast decoupled method* needs 5 to 16 iterations, each of them very cheap since the two constant matrices are factorised only once. On large networks its total time becomes com-

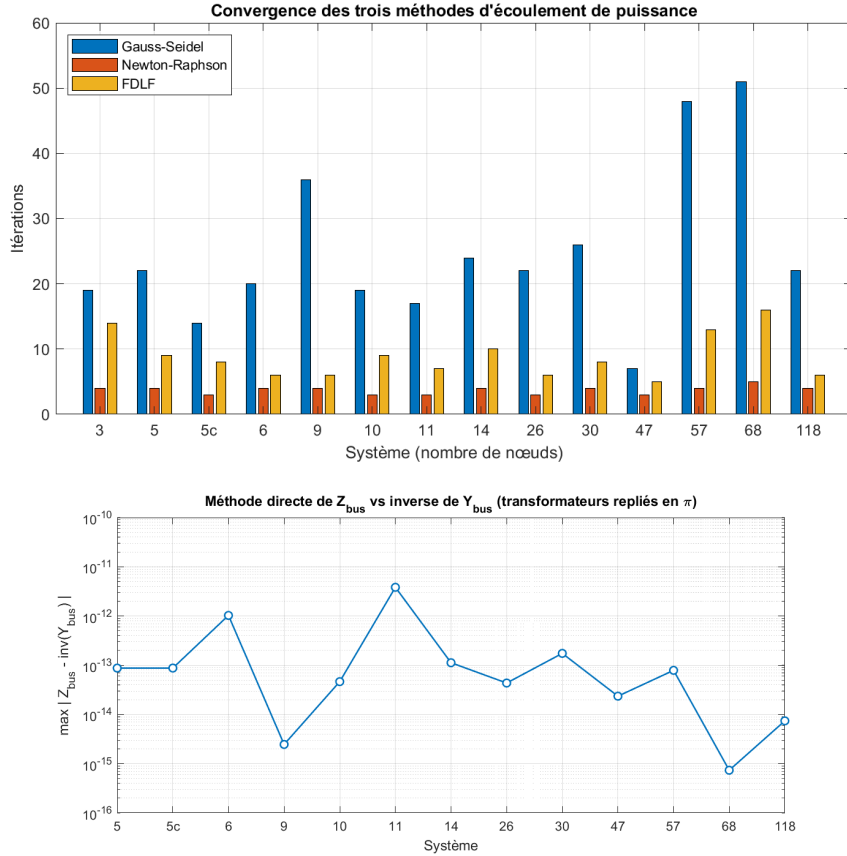


Figure 2. Top: iteration count of the three methods on the fourteen systems. Bottom: deviation $\max |Z_{bus} - Y_{bus}^{-1}|$ on a logarithmic scale for every network with a ground reference, all far below the numerical threshold of 10^{-10} .

petitive with Newton–Raphson: 12 ms against 20 ms on the 118-bus network. This is exactly the trade-off the method was designed for.

- *Gauss–Seidel* shows linear convergence, with an iteration count that grows with size and conditioning, from 7 to 51 iterations, each iteration remaining extremely simple. The 47-bus case, which converges in only 7 iterations, is a reminder that conditioning rather than size alone governs this method.

The three methods agree to better than 2×10^{-4} p.u. in voltage magnitude across all systems, which constitutes their mutual validation. Finally, the losses computed on the IEEE 118 network, 131.96 MW, match the reference value of the literature, close to 132 MW.

6.3 Detailed solution: IEEE 14-bus network

Table 4 gives the complete solution of the IEEE 14 network obtained by Newton–Raphson in 4 iterations, with losses of 13.39 MW.

Discussion. The voltage profile remains within $[1.010, 1.090]$ p.u., that is, inside the usual operating limits of ± 5 to ± 10 %. The minimum occurs at bus 3, which is a PV bus held at its set-point of 1.010 p.u. The electrically weakest point of the network, however, is bus 14, the furthest from the sources, with $|V| = 1.0343$ p.u. and $\theta = -16.04^\circ$: this is where the voltage drops and phase shifts accumulate along the path from the slack bus.

Table 4. IEEE 14-bus network: load-flow solution obtained by Newton–Raphson.

Bus	Type	$ V $ (p.u.)	θ (°)	P_L (MW)	Q_L (MVar)	P_g (MW)	Q_g (MVar)
1	slack	1.0600	0.000	0.0	0.0	232.39	−16.78
2	PV	1.0450	−4.981	21.7	12.7	40.00	42.74
3	PV	1.0100	−12.719	94.2	19.0	0	23.60
4	PQ	1.0183	−10.318	47.8	−3.9	0	0
5	PQ	1.0200	−8.781	7.6	1.6	0	0
6	PV	1.0700	−14.242	11.2	7.5	0	13.14
7	PQ	1.0608	−13.357	0	0	0	0
8	PV	1.0900	−13.357	0	0	0	18.06
9	PQ	1.0541	−14.935	29.5	16.6	0	0
10	PQ	1.0495	−15.098	9.0	5.8	0	0
11	PQ	1.0561	−14.800	3.5	1.8	0	0
12	PQ	1.0550	−15.096	6.1	1.6	0	0
13	PQ	1.0501	−15.174	13.5	5.8	0	0
14	PQ	1.0343	−16.041	14.9	5.0	0	0

The slack bus supplies 232.39 MW for a total load of 259 MW, the remaining 40 MW coming from bus 2. Active losses amount to 13.39 MW, or 4.92 % of generation, a consistent figure for a test network of this size whose lines are represented with their actual resistance.

These values coincide with the published reference solution of the IEEE 14 case, which validates the implementation on a network containing both tap-changing transformers and PV buses liable to saturate in reactive power.

6.4 Transient stability and critical clearing time

Figure 3 shows the response of the 6-bus, three-machine system to a solid fault at bus 6 cleared at $t_{cr} = 0.2$ s by opening line 4–6. The relative rotor angles oscillate and remain bounded: the system is stable. The initial rotor angles, 8.97° , 12.03° and 13.19° , come from the converged pre-fault state.

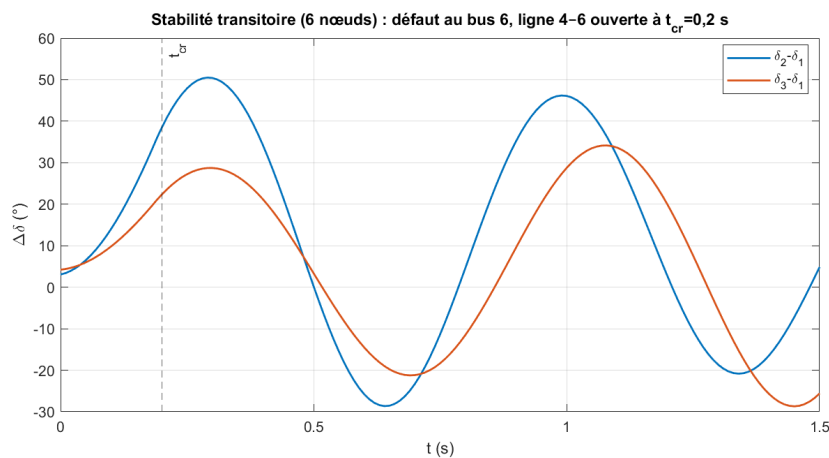


Figure 3. Transient stability of the 6-bus system: relative rotor angles $\delta_i - \delta_1$ after a fault at bus 6 cleared at $t_{cr} = 0.2$ s by opening line 4–6.

The practical value of the tool lies in the parametric study of the *critical clearing time*. The

same fault is simulated for increasing values of t_{cr} (Table 5 and Figure 4).

Table 5. 6-bus system, fault at bus 6: influence of the clearing time on the maximum angular excursion.

t_{cr} (s)	Max. angular deviation (°)	Verdict
0.10	28.0	stable
0.20	50.5	stable
0.30	79.7	stable
0.35	95.8	stable
0.40	113.0	stable
0.45	134.8	stable, close to the limit
0.50	322.7	unstable , loss of synchronism

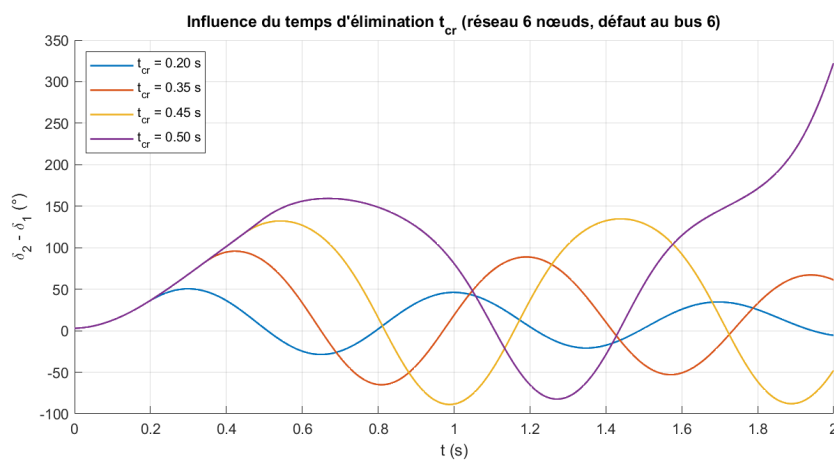


Figure 4. Relative angle $\delta_2 - \delta_1$ for different clearing times. The longer the fault persists, the larger the angular excursion of the first swing, up to the loss of synchronism between 0.45 and 0.50 s.

Discussion. The angular excursion of the first swing grows monotonically with t_{cr} , from 28° at 0.10 s to 135° at 0.45 s. The physical interpretation is direct: during the fault the transmissible electrical power collapses while the mechanical power remains constant, so the machines accelerate and accumulate kinetic energy in proportion to the fault duration. After clearing, that energy must be returned to the network; if it exceeds what the post-fault power–angle characteristic can absorb, the angle keeps growing and synchronism is lost. This is exactly the statement of the equal-area criterion.

Between $t_{cr} = 0.45$ s and $t_{cr} = 0.50$ s the system switches from stability to loss of synchronism, the angular deviation exceeding 180° divergently. The **critical clearing time is therefore between 0.45 and 0.50 s** for this configuration. The abrupt jump in the measured value, from 135° to 323° for a change of only 50 ms, is characteristic: loss of synchronism is not a gradual degradation but a switch.

This kind of study, immediate with the tool, is precisely what sizes the protection settings of a real network: the actual clearing time of the protection must remain below the critical time, with a margin sufficient to cover modelling uncertainties and more severe fault cases.

7 Conclusion

The toolbox developed covers the essentials of power system analysis: the construction of the $\mathbf{Y}_{\text{bus}}$ and $\mathbf{Z}_{\text{bus}}$ matrices, load flow by three methods and transient stability, all driven by a graphical interface and backed by automated scripts that make the results reproducible.

The results obtained on fourteen systems from 3 to 118 buses establish three points. The $\mathbf{Z}_{\text{bus}}$ built by branch addition agrees with $\mathbf{Y}_{\text{bus}}^{-1}$ to machine precision, including networks with tap-changing transformers and parallel lines, which validates each construction against the other. The three load-flow methods agree with one another and with the reference solutions of the IEEE networks. Finally, the tool supports parametric transient stability studies, the critical clearing time being located between 0.45 and 0.50 s on the 6-bus system studied.

Methodologically, the expected theoretical behaviour of the three numerical methods is recovered quantitatively: quadratic convergence of Newton–Raphson, cheap but more numerous iterations for the fast decoupled method, and linear, conditioning-sensitive convergence of Gauss–Seidel. Implementing all three rather than one has a double value: pedagogical, in making visible convergence properties that remain abstract until they are measured, and practical, since their mutual agreement is the principal means of verification available in the absence of an analytical solution.

The limitations of the tool are those of the models adopted, and they should be stated plainly. A ground-referred $\mathbf{Z}_{\text{bus}}$ does not exist for a network devoid of any shunt element, which the tool detects and reports rather than concealing. The reactive limits of PV buses are not enforced by the fast decoupled solver, which accounts for small discrepancies at saturated buses. The stability model is the classical one, with constant internal EMF and Euler integration, which restricts it to first-swing analysis: neither voltage regulation, nor governor action, nor machine damping is represented, and a study of oscillations over several seconds would require a higher-order machine model and a higher-order integrator. Subject to those restrictions, the whole constitutes a sound basis for learning and experimentation in power system analysis.

References

- [1] G. W. Stagg and A. H. El-Abiad, *Computer Methods in Power System Analysis*. New York: McGraw-Hill, 1968.
- [2] H. Saadat, *Power System Analysis*, 3rd ed. PSA Publishing, 2010.
- [3] P. Kundur, *Power System Stability and Control*. New York: McGraw-Hill, 1994.
- [4] B. Stott and O. Alsac, “Fast decoupled load flow,” *IEEE Transactions on Power Apparatus and Systems*, vol. PAS-93, no. 3, pp. 859–869, 1974.
- [5] W. F. Tinney and C. E. Hart, “Power flow solution by Newton’s method,” *IEEE Transactions on Power Apparatus and Systems*, vol. PAS-86, no. 11, pp. 1449–1460, 1967.
- [6] J. J. Grainger and W. D. Stevenson, *Power System Analysis*. New York: McGraw-Hill, 1994.
- [7] P. M. Anderson and A. A. Fouad, *Power System Control and Stability*, 2nd ed. Piscataway: IEEE Press, 2003.

A Built-in test systems

Table 6. The fourteen test systems available in the toolbox.

System	N	Notable feature	Slack bus
3	3	no shunt: $\mathbf{Y}_{\text{bus}}$ singular	1
5	5	elementary network	1
5cours	5	course variant	1
6	6	3 machines, basis of the stability study	1
9	9	tap-changing transformers	1
10	10	tap-changing transformers	1
11	11	no transformer	1
14	14	IEEE 14, transformers and saturable PV buses	1
26	26	transformers	1
30	30	IEEE 30	1
47	47	slack bus not at bus 1	9
57	57	IEEE 57	1
68	68	NETS–NYPS aggregate, heavily loaded	65
118	118	IEEE 118, parallel lines	69

B File organisation

Table 7. Organisation of the MATLAB scripts.

File	Role
Main.m	entry point, launches the interface
programme.m	construction of the App Designer interface
Y_bus.m	nodal admittance matrix constructor
Z_bus.m	direct construction of $\mathbf{Z}_{\text{bus}}$ by branch addition
gauss_seidel.m	load flow by Gauss–Seidel
newton_raphson.m	load flow by Newton–Raphson
fdlf.m	fast decoupled load flow
st.m	transient stability analysis
system<N>bus.m	definition of the fourteen test systems
test_core.m, test_st.m	automated execution of all cases
report/make_results.m, make_figs.m	generation of tables and figures